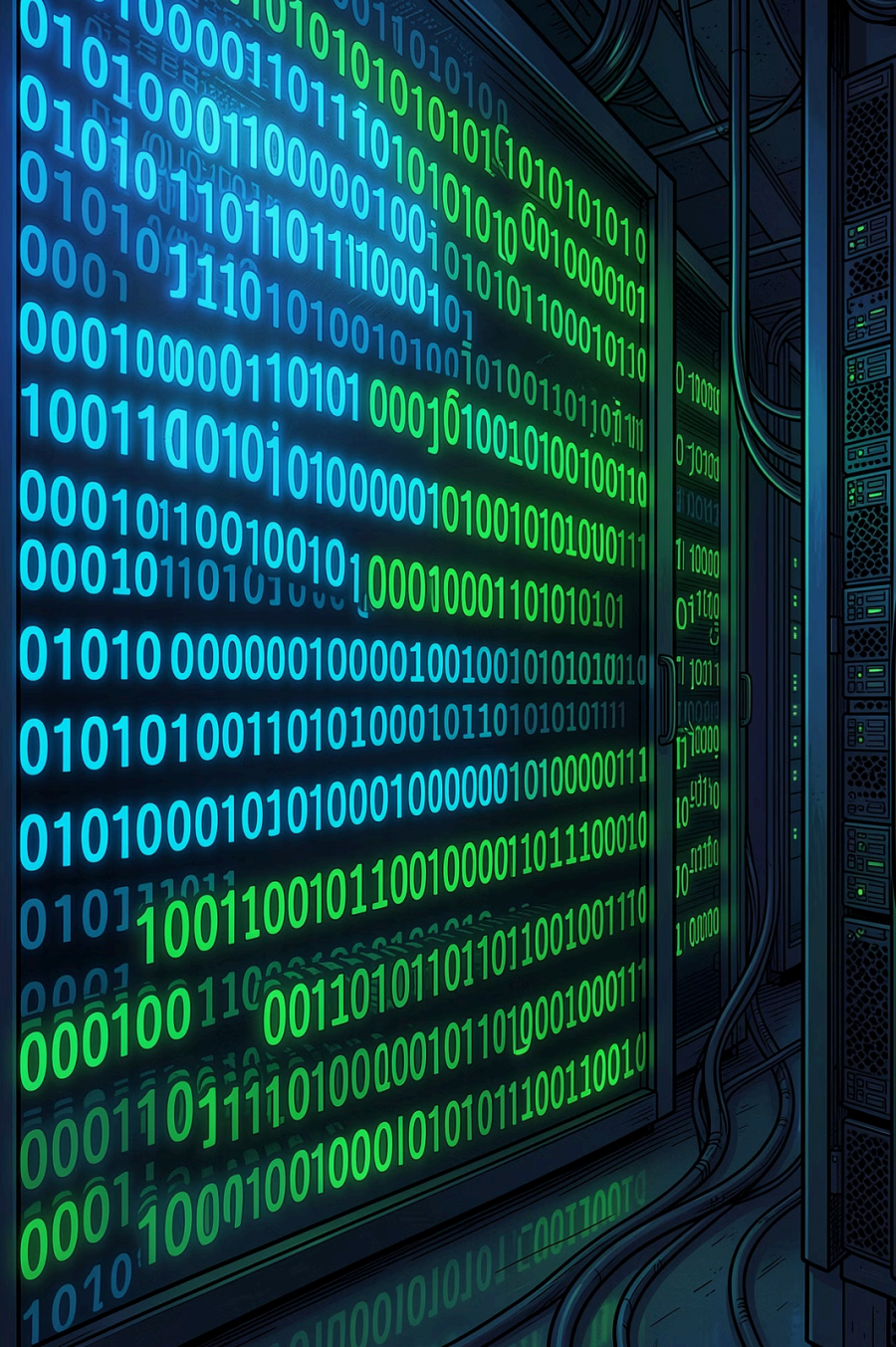




# Manipulação de Bits

- Representação binária
- Operadores bit a bit
- Conjuntos com inteiros

# Números em Binário

Um número binário é formado por potências de 2. Cada bit vale uma potência diferente, da direita para a esquerda.

## Decimal → Binário

$$13 = 8 + 4 + 1$$

Logo,  $13 = 1101$  em binário.

Marcamos os bits correspondentes a  $2^3$ ,  $2^2$  e  $2^0$ .

## Binário → Decimal

$$1101 = 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

Resultado: 13.

Basta somar os valores das posições onde o bit é 1.

## n bits → $2^n$ valores

Com **n** bits, cada posição pode ser 0 ou 1.

Por isso, o total de combinações possíveis é  $2^n$ .

Exemplo: 4 bits representam  $2^4 = 16$  valores, de 0000 (0) até 1111 (15).

# Visualizando Bits



Imagine interruptores de luz

Cada bit é um interruptor: **1 = ligado**, **0 = desligado**. Uma sequência de interruptores representa um número.

0000 = 0

Todos desligados

1010 = 10

2º e 4º ligados

1111 = 15

Todos ligados

# Complemento de Dois

O **complemento de dois** é o método mais usado para representar números negativos em binário. Ele simplifica as operações aritméticas porque soma e subtração passam a funcionar de forma uniforme no hardware.

## Como converter um número positivo

1. Inverta todos os bits
2. Some 1 ao resultado

Esse processo gera a representação do número negativo correspondente.

## Exemplo com 5

| Decimal | Binário 4 bits | Resultado           |
|---------|----------------|---------------------|
| 5       | 0101           | positivo            |
| -5      | 1011           | complemento de dois |

Passo a passo para  $5 \rightarrow -5$  em 4 bits:

1.  $5 = 0101$
2. Inverta os bits:  $1010$
3. Some 1:  $1011$

## Faixa representável com n bits

Com **n bits**, o complemento de dois representa valores de:

$$-2^{n-1} \text{ até } 2^{n-1} - 1$$

Exemplos:

- 4 bits: de -8 a 7
- 8 bits: de -128 a 127
- 16 bits: de -32768 a 32767

**i** O bit mais à esquerda é o **bit de sinal**: 0 indica número não negativo e 1 indica número negativo.

# Operadores Bit a Bit

## AND &

Retorna 1 se **ambos** os bits forem 1. Verifica se um bit está ativo:

$x \& (1 \ll k)$

## OR |

Retorna 1 se **ao menos um** bit for 1. Ativa um bit:

$x \mid (1 \ll k)$

## XOR ^

Retorna 1 se os bits forem **diferentes**. Inverte um bit:

$x \wedge (1 \ll k)$

## NOT ~

Inverte **todos** os bits. Use com cuidado em inteiros com sinal.

## Shift Esquerdo <<

Multiplica por potência de 2.

$x \ll k = x \times 2^k$

## Shift Direito >>

Divide por potência de 2.

$x \gg k = x / 2^k$

# Operações Práticas com Bits

O c++ oferece funções nativas para resolver operações comuns com bits de forma muito eficiente ( $O(1)$ ).

`__builtin_clz(x)`

Conta os **zeros à esquerda** até o primeiro bit 1 mais significativo.

Para `x = 5328`, o resultado é **19**.

`__builtin_ctz(x)`

Conta os **zeros à direita** até o primeiro bit 1 menos significativo.

Para `x = 5328`, o resultado é **4**.

`__builtin_popcount(x)`

Conta quantos bits têm valor **1** em `x`.

Para `x = 5328`, o resultado é **5**.

`__builtin_parity(x)`

Retorna a **paridade** dos bits 1: **1** se a quantidade de bits ativos for ímpar, **0** se for par.

Para `x = 5328`, o resultado é **1**.

❗ Exemplo com `x = 5328`:

```
00000000000000000000101001101000
```

```
__builtin_clz(x) = 19
```

```
__builtin_ctz(x) = 4
```

```
__builtin_popcount(x) = 5
```

```
__builtin_parity(x) = 1
```

Essas funções também têm versões para long long, com o sufixo ll: `__builtin_clzll`, `__builtin_ctzll`, `__builtin_popcountll` e `__builtin_parityll`.

# Representando Conjuntos com Inteiros

Cada bit de um inteiro indica a **presença ou ausência** de um elemento no conjunto.

## Regra

Se o  $k$ -ésimo bit é 1, o elemento  $k$  pertence ao conjunto. Se é 0, não pertence.

## Exemplo

Para elementos {0, 1, 2, 3}: o inteiro 1101 em binário representa o conjunto {0, 2, 3} (bits 0, 2 e 3 ativos).

## Vantagem

Subconjuntos de universos até 32 ou 64 elementos cabem em um único int ou long long.



# Operações de Conjuntos com Bits

| Operação              | Código C++                           | Descrição           |
|-----------------------|--------------------------------------|---------------------|
| União $A \cup B$      | <code>A   B</code>                   | Elemento em A ou B  |
| Interseção $A \cap B$ | <code>A &amp; B</code>               | Elemento em ambos   |
| Diferença $A - B$     | <code>A &amp; ~B</code>              | Em A mas não em B   |
| Pertence?             | <code>A &amp; (1 &lt;&lt; k)</code>  | Não-zero = presente |
| Adicionar k           | <code>A   (1 &lt;&lt; k)</code>      | Insere elemento k   |
| Remover k             | <code>A &amp; ~(1 &lt;&lt; k)</code> | Remove elemento k   |

## Por que isso é eficiente?

Todas essas operações são executadas em **O(1)** — tempo constante. Isso torna bitmasks ideais para problemas de subconjuntos e programação dinâmica.

- ✔ Iterando sobre todos os  $2^n$  subconjuntos de um universo de n elementos com um simples loop:

```
for(int s=0; s<(1<<n); s++)
```